



Introduction to jQuery

& Introducing LINQ

*jQuery Effects • LINQ to Objects • LINQ to SQL
Query Syntax • Projection • Filtering • Join*

What is jQuery?

Key Features

- Fast, small JavaScript library
- Simplifies HTML manipulation
- Easy DOM traversal
- Event handling made simple
- AJAX calls with fewer lines
- Cross-browser compatible

Basic Syntax

`$(selector).action()`

\$ = jQuery function

selector = find HTML element

action() = perform action

Example:

```
$("#btn").hide();
```

jQuery Effects

Hide / Show

```
.hide() .show()  
.toggle()
```

*Control element
visibility*

Fade

```
.fadeIn()  
.fadeOut()  
.fadeToggle()
```

*Smooth opacity
transitions*

Slide

```
.slideDown()  
.slideUp()  
.slideToggle()
```

*Slide elements up or
down*

Animate

```
.animate({  
  opacity: 0.5  
})
```

Custom CSS animations

💡 Speed: "slow", "fast", or milliseconds — e.g., `.hide("slow")` or `.fadeIn(1000)`

jQuery Effects – Callback & Chaining

Callback Functions

```
$("#p").hide("slow", function() {  
    alert("The paragraph is now hidden!");  
});
```

→ A callback runs after the effect is completed.

Method Chaining

```
$("#box").slideDown("slow").slideUp("slow").fadeIn();
```

→ Chaining allows multiple effects in one line on the same element.

✓ Effects run in order — each starts after the previous one finishes!

Introducing LINQ

Language Integrated Query

What is LINQ?

LINQ adds query capabilities directly into C#/VB.NET. Write SQL-like queries inside your code.

Why use LINQ?

Cleaner, readable code. Less boilerplate. Works with arrays, lists, databases, XML, and more.

Where is it used?

LINQ to Objects, LINQ to SQL, LINQ to XML, LINQ to Entities (EF).

LINQ to Objects

Query any in-memory collection (arrays, lists, etc.) using LINQ.

Without LINQ

```
List<int> evens = new List<int>();  
foreach (int n in numbers)  
{  
    if (n % 2 == 0)  
        evens.Add(n);  
}
```

With LINQ ✓

```
var evens = numbers  
    .Where(n => n % 2 == 0)  
    .ToList();
```

Common Methods:

.Where()

Filter elements

.Select()

Transform / project

.OrderBy()

Sort ascending

.FirstOrDefault()

Get first match

LINQ to SQL

Map C# classes to database tables. Write queries in C# — no raw SQL needed!

1

Create DataContext

Connect to your database via
DataContext class

2

Map Tables

C# classes represent DB tables
using [Table] attribute

3

Write Query

Use LINQ syntax to query the
mapped objects

4

Execute

LINQ translates your query
into SQL automatically

Example

```
DataContext db = new DataContext(connectionString);  
var students = from s in db.GetTable<Student>()  
                where s.Age > 18  
                select s;
```

LINQ Query Syntax

Two ways to write LINQ queries — both produce the same result:

Query Syntax (SQL-like)

```
var result =  
    from s in students  
    where s.Score > 80  
    orderby s.Name  
    select s;
```

Method Syntax (Lambda)

```
var result = students  
    .Where(s => s.Score > 80)  
    .OrderBy(s => s.Name);
```

Query Syntax Keywords:

from

where

select

orderby

group by

join

let

into

LINQ Operation: Projection

Projection = Selecting / transforming specific fields from a collection.

Use the **select** keyword to project data

```
// Select only names
var names = from s in students select s.Name;

// Select new anonymous object
var info = from s in students
           select new { s.Name, s.Score };
```

Same using Method Syntax

```
var names = students.Select(s => s.Name);
var info  = students.Select(s => new { s.Name, s.Score });
```

 Use projection to get only what you need — reduces memory usage!

LINQ Operation: Filtering

Filtering = Selecting only elements that match a condition.

Use the where keyword / .Where() method

```
// Query Syntax  
var passed = from s in students  
             where s.Score >= 50  
             select s;
```

```
// Method Syntax  
var passed = students  
             .Where(s => s.Score >= 50);
```

Multiple Conditions

```
var topStudents = students.Where(s => s.Score >= 80 && s.Age < 25);
```

Common Operators:

== Equals

!= Not equal

> < Greater / Less

&& || AND / OR

LINQ Operation: Join

Join = Combine two collections based on a matching key (like SQL JOIN).

Students		Courses		Result	
ID	Name	StudID	Course	Name	Course
1	Hassan	1	Math	Hassan	Math
2	Ali	2	Science	Ali	Science

JOIN →

LINQ Join Code

```
var result = from s in students
              join c in courses
              on s.ID equals c.StudentID
              select new { s.Name, c.Course };
```

Summary

jQuery Effects

hide/show, fade, slide, animate, callback, chaining

LINQ to Objects

Query in-memory collections like List<T>, arrays

Query Syntax

SQL-like: from ... where ... select vs Method/Lambda syntax

Filtering

where — return only elements matching a condition

Introducing LINQ

Language Integrated Query — query data inside C# code

LINQ to SQL

Query databases using C# classes mapped to DB tables

Projection

select — pick/transform specific fields

Join

join ... on ... equals — combine two collections by key